Edge Intelligence Laboratory --7 **Oviya R**

25MML0007

1. The file name is **real_world_demo_video.mp4** which is a video file that is **uploaded** in google colab and the video path is defined

2. **MobileNetV2** trained on **ImageNet dataset (1000 object classes).**

MobileNetV2 is ideal for quick demo video classification.

3.**Extracts Frames from Video**

Using OpenCV

- It Opens video
- Samples upto 20 frames evenly
- Resizes to 224×224
- Converts BGR to RGB
- Returns numpy array

4. **preprocess_input()** scales pixel values to range expected by MobileNetV2.

5.**np.mean(predictions, axis=0)**

This gives the average object probability across all frames which becomes the final video prediction

6.Finally it Outputs top 5 classes

```python
from google.colab import files
uploaded = files.upload()
```

```
Choose Files real_world_..._video.mp4
real_world_demo_video.mp4(video/mp4) - 433292 bytes, last modified: 19/2/2026 - 100% done
Saving real_world_demo_video.mp4 to real_world_demo_video (1).mp4
```

```python
video_path = "/content/real_world_demo_video.mp4"
```

```python
def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    # Preprocess frames
    frames = preprocess_input(frames)

    # Predict
    predictions = model.predict(frames, verbose=0)

    # Average predictions
    avg_pred = np.mean(predictions, axis=0)

    # Decode top 5
    top_pred = decode_predictions(
        np.expand_dims(avg_pred, axis=0), top=5
    )

    print("\nTop Predictions:")
    print("-" * 40)

    for pred in top_pred[0]:
        print(pred[1], ":", round(pred[2] * 100, 2), "%")

    print("\nFinal Prediction:", top_pred[0][0][1])


# ✅ Correct Colab path
video_path = "/content/real_world_demo_video.mp4"
classify_video(video_path)
```

```
Extracted frames: 20

Top Predictions:
----------------------------------------
digital_clock : 63.84 %
traffic_light : 9.13 %
switch : 1.43 %
computer_keyboard : 1.32 %
matchstick : 0.85 %

Final Prediction: digital_clock
```

Error

Processing video: /mnt/data/real_world_demo_video.mp4

ValueError Traceback (most recent call last) /tmp/ipython-input-570675781.py in <cell line: 0>()

33 # ---------- 34 video_path = "/mnt/data/real_world_demo_video.mp4" ---> 35 classify_video(video_path) 1 frames /tmp/ipython-input-4236132701.py in extract_frames(video_path, max_frames, img_size) 11 12 if not cap.isOpened(): ---> 13 raise ValueError("Error: Cannot open video file") 14 15 frames = []

ValueError: Error: Cannot open video file

Also this error came when the video cannot accessed

I used the below code to load the video into the correct location

video_path = "/content/real_world_demo_video.mp4"

classify_video(video_path)

```
video_path = "/content/real_world_demo_video.mp4"
classify_video(video_path)
```

```
Extracted frames: 20

Before Preprocessing:
Min pixel value: 0.0
Max pixel value: 255.0
Sample pixel [100,100]: [233. 206. 133.]

After Preprocessing:
Min pixel value: -1.0
Max pixel value: 1.0
Sample pixel [100,100]: [0.827451   0.6156863  0.04313731]

Top Predictions:
----------------------------------------
digital_clock : 63.84 %
traffic_light : 9.13 %
switch : 1.43 %
computer_keyboard : 1.32 %
matchstick : 0.85 %

Final Prediction: digital_clock
```

```python
frames = extract_frames("/content/real_world_demo_video.mp4")
print(frames.shape)
```

```
Extracted frames: 20
(20, 224, 224, 3)
```

```python
frames = extract_frames("/content/real_world_demo_video.mp4")

print("Shape:", frames.shape)

print("\nMatrix of First Image:\n")
print(frames[0])
```

```
Extracted frames: 20
Shape: (20, 224, 224, 3)

Matrix of First Image:

[[[233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]
  ...
  [233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]]

 [[233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]
  ...
  [233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]]

 [[233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]
  ...
  [233. 206. 133.]
  [233. 206. 133.]
  [233. 206. 133.]]

 ...

 [[ 46.  48.  45.]
  [ 46.  48.  45.]
  [ 46.  48.  45.]
  ...
  [ 46.  48.  45.]
  [ 46.  48.  45.]
  [ 46.  48.  45.]]

 [[ 46.  48.  45.]
  [ 46.  48.  45.]
  [ 46.  48.  45.]
  ...
```

Task-2

1. This is also a video dataset which is uploaded initially

2.After the video is loaded, number of frames are checked

```
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)

2398.0
```

3. The height and width of the video is checked and **this video is a HD Quality video** because **1280*720**

4. Then frames per second(fps) is also considered

```
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')

FPS : 59.94
```

5. The first frame of the image is plotted using **matplotlib**, but it uses only **RGB format**. The **OpenCV** loads colour in **BGR format**

Red channels swapped with blue

Original image

While running I found that mot_labels.csv is not downloaded so it gave file not found error so I downloaded it and rerun it

```
labels = pd.read_csv('mot_labels.csv',low_memory=False)
##labels.head()
#video_labels = (labels.query('videoName == "026c7465-309f6d33"').reset_index(drop=True).copy())
#video_labels["video_frame"] = (video_labels["frameIndex"] * 11.9).round().astype("int") #Video is at 60Hz but the labellers did only 5 Hz

---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-1597131570.py in <cell line: 0>()
----> 1 labels = pd.read_csv('mot_labels.csv',low_memory=False)
      2 ##labels.head()
      3 #video_labels = (labels.query('videoName == "026c7465-309f6d33"').reset_index(drop=True).copy())
      4 #video_labels["video_frame"] = (video_labels["frameIndex"] * 11.9).round().astype("int") #Video is at 60Hz but the labellers did only 5 Hz

                              4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'mot_labels.csv'
```

```
import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:                                        #frame is divisible by 100 - Note the modulo operator
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()                                              #automatically adjusts the spacing between the plots
plt.show()
cap.release()
```

```python
print(labels["category"].value_counts())
```

```
category
car              394391
pedestrian        67862
truck             22812
bus               10694
bicycle            3823
other vehicle      3756
rider              3307
motorcycle         2277
other person        782
trailer             376
train               156
Name: count, dtype: int64
```